

From AI agents to data engineering

DeNederlandscheBank

EUROSYSTEM

Michiel Nijhuis (EBO_DSH)

Content

1. From AI agents to data engineering

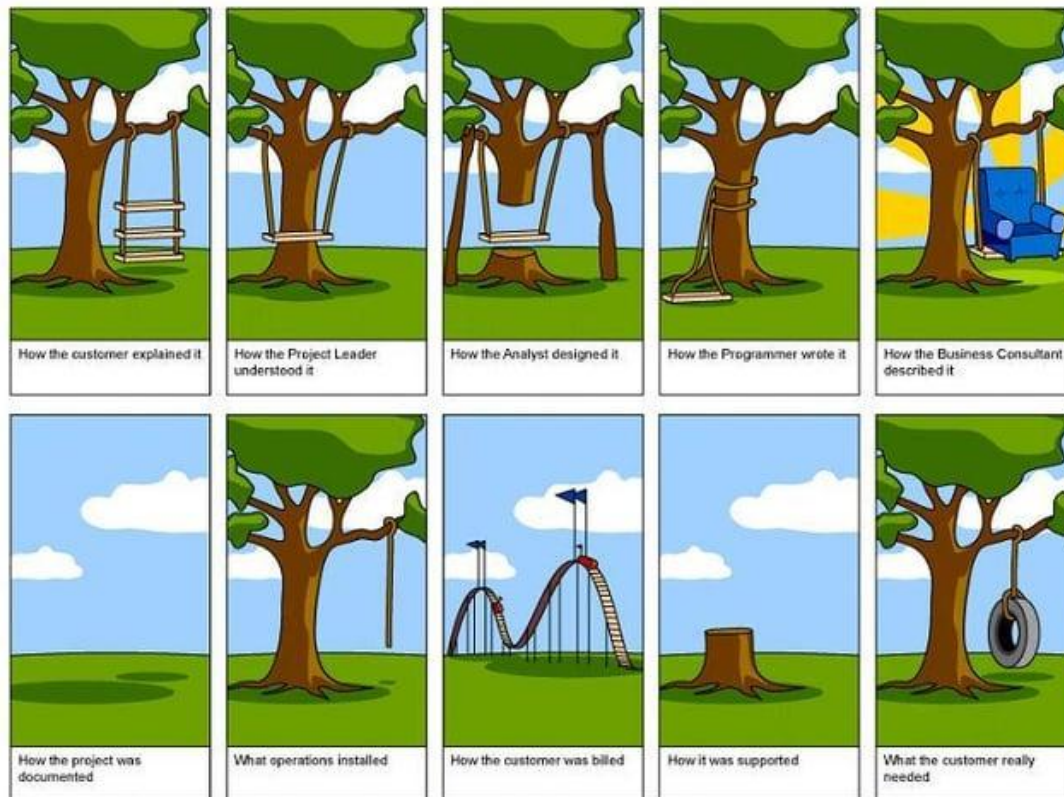
- Why reliable data flows are critical for agent-based systems

2. Key data engineering concepts

- Data ownership, data locations, and data access patterns
- Reproducibility, versioning, and data lineage

3. ML workflows in practice

- Setting up an end-to-end ML workflow
- Practical with setting up a workflow





Reproducibility, versioning, and data lineage

Reproducibility:

A workflow is reproducible when the same inputs, code, configuration, and data snapshot produce the same result. This enables debugging, auditing, collaboration, and confidence in why an output changed.

Versioning:

Versioning tracks changes across code, data, models, and settings over time. It lets teams compare states, restore earlier versions, and understand whether outcomes shifted because of code updates or data changes.

Data lineage:

Data lineage shows how data moves from source to destination and what transformations occur along the way. It clarifies origin, dependencies, and downstream impact, supporting faster issue resolution, governance, and trustworthy analytics and machine learning.

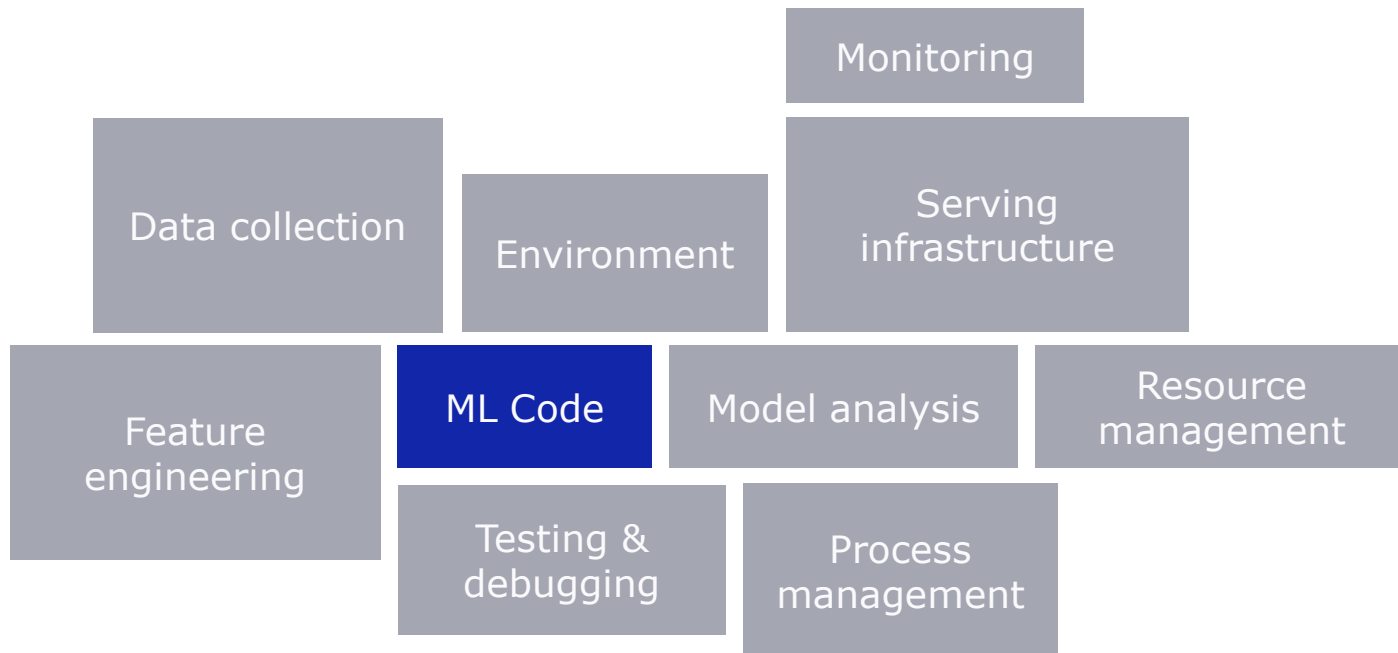
Data quality dimensions for agent systems

- **Accuracy:** Data matches the real entity or event. It matters because wrong inputs create false context, misleading features, and incorrect outputs.
- **Completeness:** Required fields, events, labels, and metadata are present. It matters because missing information weakens retrieval and leads to partial decisions.
- **Consistency:** Definitions and formats stay aligned across systems and teams. It matters because mixed signals reduce reliability and trust.
- **Timeliness:** Data is current and refreshed at the right cadence. It matters because stale inputs make responses lag behind reality.
- **Traceability:** Teams can track source, transformations, and versions. It matters because auditability, debugging, and reproducibility depend on it.

Data and AI agents

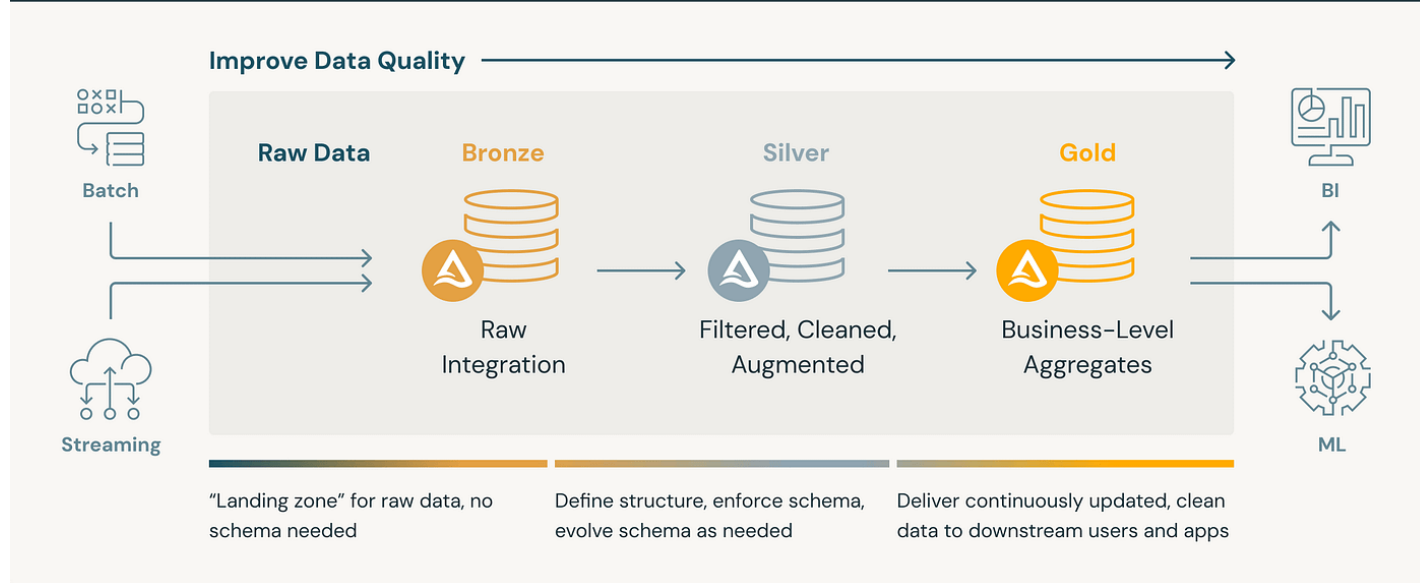
- AI agents always provide answers, even if the data is bad
- Tabular data easier to validate then text
- How do you handle data u use?
- How do you track results from different model runs?
- How do you share models?

AI is just part of the system



Data ETL

Building reliable, performant data pipelines with  **DELTA LAKE**



Feature engineering and dataset preparation

- 1. Define the prediction or decision context** Start from exact question; target, time window, evaluation boundaries set record inclusion; ambiguity adds noise, misleads.
- 2. Prepare clean and usable inputs** Normalize, handle missing, encode categories, aggregate, align; avoid future leakage; disciplined prep > model complexity.
- 3. Create and review candidate features** Prioritize signal, stability, interpretability, production availability; ensure features consistent, maintainable, trustworthy post-deployment.
- 4. Package datasets for repeatable use** Split data, document assumptions, version datasets, reuse prep logic; strong prep enables repeatability, collaboration, reliable learning.

Metadata

Storage patterns and access trade-offs

- **Raw data lake / landing zone:** Best for replay, audit, and flexible downstream processing. Preserves source fidelity, but needs strong governance to stay navigable and trustworthy.
- **Curated warehouse / analytical layer:** Best for standardized reporting and trusted metrics. Improves consistency and discoverability, but adapts more slowly to new structures or experimentation.
- **Feature-oriented serving layer:** Best for reusable ML inputs across training and inference. Promotes development-to-production consistency, but adds operational, versioning, and freshness complexity.
- **Vector / retrieval-oriented store:** Best for semantic search, retrieval augmentation, and agent context assembly. Enables unstructured access, but depends heavily on chunking, metadata, refresh, and retrieval design.
- **Hybrid architecture:** Best when reporting, ML, and agent workflows must coexist. Matches storage to workload, but increases integration and governance demands.

Data modeling basics

Why modeling matters

Data modeling creates a shared structure for analytics and operational use. It improves consistency across teams and systems, and makes data easier to query, trust, and reuse.

Common structures

Normalized models organize data into related tables to reduce duplication and keep updates accurate. Dimensional models are built for analysis, usually with facts for measurable events and dimensions for descriptive context such as customer, product, or date.

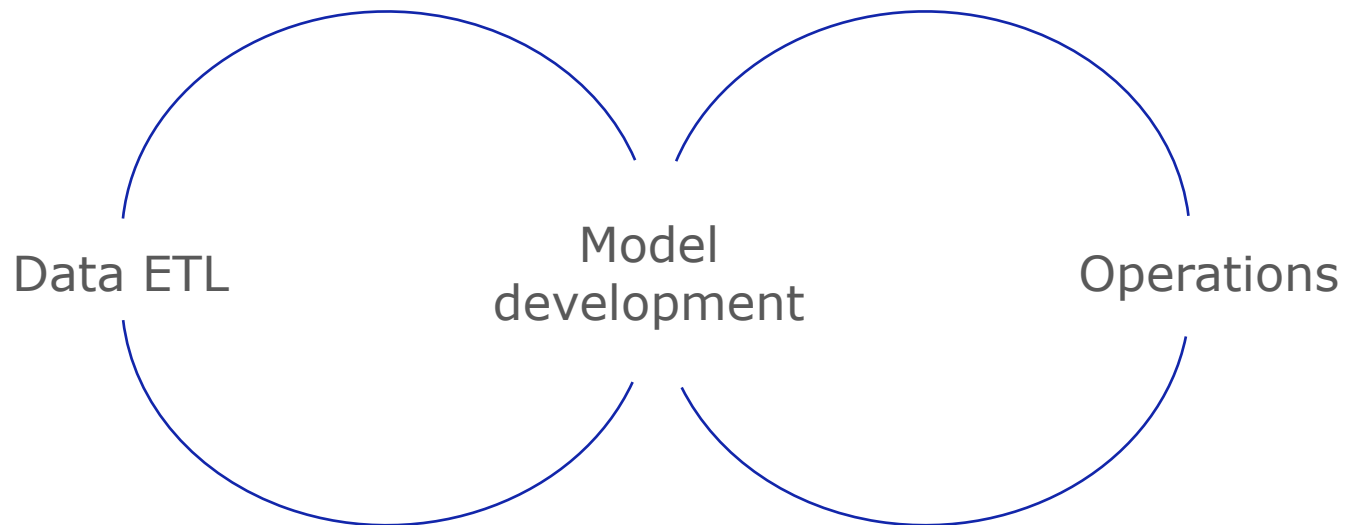
Practical design principles

Choose clear business keys, define grain early so each record has one meaning, reduce ambiguity in names, and design models that are easy to maintain as reporting and process needs evolve.

Setting up an end-to-end ML workflow

- 1. Define workflow scope & success:** Specify use case, users, outputs, data, metrics.
- 2. Data ingestion & storage:** Ingest sources; separate raw, transformed, production assets.
- 3. Transformation & feature prep:** Repeatable cleaning, standardization, joins, feature logic.
- 4. Model training & tracking:** Log experiments, data versions, parameters, evaluation.
- 5. Deployment, monitoring & governance:** Define serving, output storage, integration, failure handling; monitor pipeline/model quality; enforce ownership, access, versioning, audit.

Models are part of a development cycle



Overview on MLflow

- Open-source platform for managing end-to-end machine learning lifecycle developed by Databricks
- Four primary functions:
 - Tracking experiments to record and compare parameters and results
 - Packaging ML code in a reusable, reproducible form
 - Managing and deploying models
 - Model registry
- Has both a Python as well as an R API (also Java)

Let's try it yourself

DeNederlandscheBank

EUROSYSTEEM

Michiel Nijhuis (EBO_DSH)